

## **PROGRAMMĒŠANA**

# **KVALIFIKĀCIJAS DARBS PATVERSMES AUTOMATIZĀCIJAS PLATFORMA**

**DARBA AUTORS**

**ALANS ARZUMANJANS**

**DARBA VADĪTĀJS**

**VLADISLAVS MEDVEDEVS**

**RĪGA, 2026**

## SATURA RĀDĪTĀJS

|                                                                     |    |
|---------------------------------------------------------------------|----|
| SAĪSINĀJUMU UN SIMBOLU SARAKSTS .....                               | 4  |
| ANOTĀCIJA .....                                                     | 6  |
| IEVADS .....                                                        | 7  |
| 1. UZDEVUMA FORMULĒJUMS .....                                       | 9  |
| 2. PRASĪBU SPECIFIKĀCIJA .....                                      | 11 |
| 2.1. Sistēmas funkcionālās prasības .....                           | 11 |
| 2.1.1. Lietotāju reģistrācija un autentifikācija .....              | 11 |
| 2.1.2. Dzīvnieka profila izveide un redigēšana .....                | 11 |
| 2.1.3. Adopcijas pieteikuma apstrāde .....                          | 12 |
| 2.1.4. Reāllaika čats starp adoptētāju un patversmi .....           | 12 |
| 2.1.5. IoT sensoru datu monitorings un brīdinājumu ģenerēšana ..... | 13 |
| 2.2. Sistēmas nefunkcionālās prasības .....                         | 13 |
| 2.2.1. Veiktspēja .....                                             | 13 |
| 2.2.2. Drošība .....                                                | 13 |
| 2.2.3. Lietojamība .....                                            | 13 |
| 2.2.4. Saderība .....                                               | 14 |
| 3. SISTĒMAS TEHNISKAIS APRAKSTS .....                               | 15 |
| 3.1. Sistēmas arhitektūra .....                                     | 15 |
| 3.2. Līdzekļu, tehnoloģiju un valodu apraksts .....                 | 16 |
| 3.3. IoT apakšsistēma .....                                         | 19 |
| 3.4. Datu plūsma sistēmā .....                                      | 19 |
| 3.5. Drošības arhitektūra .....                                     | 20 |
| 3.6. Sistēmas mērogojamība un paplašināmība .....                   | 22 |
| 3.7. Datubāzes struktūra un ER modelis .....                        | 23 |
| 4. PROGRAMMATŪRAS TESTĒŠANA UN REZULTĀTI .....                      | 25 |
| 4.1. Testēšanas metodoloģija un pieeja .....                        | 26 |
| 4.2. Izmantotie rīki un testēšanas vide .....                       | 27 |
| 4.3. Vienību testu rezultāti .....                                  | 29 |
| 4.4. Integrācijas un API endpointu testi .....                      | 29 |
| 4.5. Drošības un veiktspējas aspekti .....                          | 29 |
| 4.6. Testu rezultātu kopsavilkums .....                             | 30 |
| SECINĀJUMI .....                                                    | 31 |

|                                                |    |
|------------------------------------------------|----|
| IESPĒJAMIEUZLABOJUMI .....                     | 32 |
| IZMANTOTĀS LITERATŪRAS UN AVOTU SARAKSTS ..... | 33 |

## SAĪSINĀJUMU UN SIMBOLU SARAKSTS

**ACID** — Atomicity, Consistency, Isolation, Durability - četras īpašības, kas garantē datubāzes transakciju uzticamību: nedalāmību, konsekvenci, izolāciju un noturīgumu [3]

**API** — Application Programming Interface - programmatūras saskarne, kas ļauj dažādām sistēmām apmainīties ar datiem un funkcijām saskaņā ar noteiktiem protokoliem [1]

**CI/CD** — Continuous Integration / Continuous Delivery - izstrādes prakse, kurā koda izmaiņas tiek automātiski pārbaudītas (CI) un izvietotas ražošanas vidē (CD), izmantojot automatizētus cauruļvadus [13]

**CO<sub>2</sub>** — oglekļa dioksīds - bezkrišains gāze, kuras koncentrācijas līmenis tiek uzraudzīts dzīvnieku telpās kā gaisa kvalitātes rādītājs

**CORS** — Cross-Origin Resource Sharing - HTTP mehānisms, kas nosaka, no kuriem domēniem pārlūkprogramma drīkst pieprasīt resursus, nodrošinot drošu starpdomēnu saziņu [1]

**CSRF** — Cross-Site Request Forgery - tīmekļa uzbrukuma veids, kurā ļaunprātīga vietne nosūta neatļautus pieprasījumus autentificēta lietotāja vārdā; sistēmā aizsargāts ar antiforgery marķieriem

**DDoS** — Distributed Denial of Service - koordinēts kiberuzbrukums, kurā liels skaits ierīču vienlaicīgi pārslogo serveri, padarot to nepieejamu; aizsardzību nodrošina Cloudflare [14]

**GIL** — Global Interpreter Lock - Python valodas mehānisms, kas ierobežo vienlaicīgu vairāku pavedienu izpildi, samazinot veiktspēju daudzpavedienu lietojumos

**HTTPS** — Hypertext Transfer Protocol Secure - HTTP protokola drošā versija, kas izmanto TLS šifrēšanu, lai aizsargātu datu pārsūtīšanu starp klientu un serveri [12]

**IoT** — Internet of Things - fizisko ierīču (sensoru, mikrokontrolleru u.c.) tīkls, kas savāc un pārsūta datus caur internetu; sistēmā realizēts ar ESP32 mikrokontrolleriem un Sensirion sensoriem [4]

**JWT** — JSON Web Token - kompakts, kriptogrāfiski parakstīts marķieris, ko izmanto lietotāja identitātes un tiesību nodošanai starp klientu un serveri bez sesiju glabāšanas serverī [10]

**ORM** — Object-Relational Mapping - programmēšanas tehnika, kas ļauj strādāt ar datubāzi, izmantojot objektorientētas klases, nerakstot tiešus SQL vaicājumus; sistēmā izmantots Entity Framework Core [1]

**REST** — Representational State Transfer - arhitektūras stils tīmekļa pakalpojumu izstrādei, kas balstās uz HTTP metodēm (GET, POST, PUT, DELETE) un bezstāvokļa saziņu starp klientu un serveri [12]

**UFW** — Uncomplicated Firewall - lietotājam draudzīgs uguns mūra pārvaldības rīks Linux operētājsistēmā, kas vienkāršo tīkla trafika filtrēšanas noteikumu konfigurēšanu

**WAF** — Web Application Firewall - specializēts uguns mūris, kas analizē HTTP/HTTPS trafiku un bloķē tīmekļa uzbrukumus (SQL injekcijas, XSS u.c.); sistēmā nodrošināts caur Cloudflare [14]

**AI** — Artificial Intelligence - datorzinātnes nozare, kas nodarbojas ar intelektuālu uzdevumu automatizāciju; sistēmā paredzēts kā nākotnes paplašinājums dzīvnieku uzvedības analīzei

**bcrypt** — kriptogrāfisks paroļu jaukšanas algoritms, kas izmanto sāļus (salt) un pielāgojamu sāļu skaitu, lai aizsargātu paroles pret brutāla spēka uzbrukumiem; sistēmā konfigurēts ar  $\geq 10$  sāļiem [11]

**CDN** — Content Delivery Network - ģeogrāfiski sadalītu serveru tīkls, kas piegādā statisko saturu (attēlus, skriptus u.c.) no lietotājam tuvākā mezgla, samazinot ielādes laiku; nodrošina Cloudflare [14]

**Cloudflare** — mākoņpakalpojumu platforma, kas darbojas kā sistēmas perimetra drošības vārteja, nodrošinot DDoS aizsardzību, WAF filtrēšanu, SSL/TLS termināciju un globālo CDN piegādi [14]

**CRUD** — Create, Read, Update, Delete - četras pamata datu pārvaldības operācijas, ko veic uz datubāzi; sistēmā realizētas caur ASP.NET Core REST API endpointiem [1]

**DI** — Dependency Injection - projektēšanas paraugs, kurā objekta atkarības tiek padotas no āras, nevis izveidotas pašā objektā; uzlabo testējamību un moduļu atdalīšanu; ASP.NET Core iebūvēts DI konteiners [1]

**Docker** — platforma programmatūras konteinerizācijai, kas ļauj iepakot lietotni ar visām atkarībām vienotā konteinerā un izvietot to jebkurā vidē bez konfigurācijas izmaiņām; sistēmā pārvaldīts ar Docker Compose [5]

**DOM** — Document Object Model - programmēšanas saskarne, kas attēlo HTML dokumentu kā koku struktūru un ļauj JavaScript dinamiski mainīt tīmekļa lapas saturu; React izmanto virtuālo DOM veiktspējas optimizācijai [2]

**ESP32** — Espressif Systems divkodolu mikrokontrolle ar integrētu Wi-Fi un Bluetooth, ko sistēmā izmanto IoT sensoru mezglos CO<sub>2</sub>, temperatūras un mitruma datu vākšanai un pārsūtīšanai uz backend API [4]

**FK** — Foreign Key — datubāzes kolonnas vērtība, kas atsaucas uz primāro atslēgu citā tabulā, nodrošinot entītiņu savstarpējo saistību integritāti PostgreSQL shēmā [3]

**HMR** — Hot Module Replacement (karstā moduļu nomaiņa izstrādes laikā)

**HTTP** — Hypertext Transfer Protocol - pamatprotokols datu apmaiņai tīmeklā, kas nosaka pieprasījumu (GET, POST u.c.) un atbilžu formātu starp klientu un serveri [12]

**JSON** — JavaScript Object Notation - viegls, cilvēklasāms datu apmaiņas formāts, ko sistēmā izmanto API atbilžu, IoT telemetrijas un MongoDB dokumentu struktūrēšanai [8]

**LINQ** — Language Integrated Query - vaicājumu valoda, kas integrēta C# valodā un ļauj veikt tipdrošus datu vaicājumus gan datubāzēm (caur Entity Framework Core), gan kolekcijām [1]

**MAC** — Media Access Control - tīkla interfeisa unikāla fiziskā adrese, ko sistēmā izmanto IoT ierīču (ESP32) identificēšanai un reģistrācijai backend datubāzē [4]

**ML** — Machine Learning - mākslīgā intelekta apakšnozare, kurā algoritmi māca no datiem bez tiešas programmēšanas; sistēmā paredzēts kā nākotnes paplašinājums dzīvnieku veselības anomāliju atklāšanai pēc IoT datiem

**MongoDB** — dokumentu orientēta NoSQL datubāze, kas uzglabā datus JSON formātā; sistēmā izmantota dzīvnieku fotoattēlu un IoT telemetrijas žurnālu glabāšanai pateicoties elastīgai shēmai un augstai rakstīšanas caurlaidībai [8]

**Nginx** — augstveiktspējas tīmekļa serveris un apgrieztais starpniekserveris, kas sistēmā nodrošina React statisko failu apkalpošanu, pieprasījumu pārsūtīšanu uz ASP.NET Core backend un drošības galvenes [14]

**PK** — Primary Key - datubāzes kolonnas unikāla vērtība, kas viennozīmīgi identificē katru ierakstu tabulā; PostgreSQL shēmā parasti izmantoti automātiski ģenerēti UUID vai secīgi veseli skaitļi [3]

**RBAC** — Role-Based Access Control - drošības modelis, kurā sistēmas resursu pieejamība tiek piešķirta atbilstoši lietotāja lomai (Administrators, Patversme, Klients), nevis individuāliem lietotājiem [1]

**React** — atvērta koda JavaScript bibliotēka, kuru izstrādājusi Meta, lietotāja saskarnes komponentu veidošanai; sistēmā izmantota frontend izstrādei kopā ar TypeScript un Vite bundleri [2]

**Redis** — atmiņā balstīta datu struktūru krātuve ar atbildes laikiem mikrosekunžu diapazonā; sistēmā izmantota API kešatmiņai, SignalR backplane sinhronizācijai, refresh token revokācijai un pieprasījumu ierobežošanai [9]

**SignalR** — ASP.NET Core bibliotēka reāllaika divvirzienu saziņai starp klientu un serveri, izmantojot WebSocket protokolu; sistēmā nodrošina IoT sensoru datu tiešraidi dashboardā un tērēšanas funkcionalitāti [7]

**SSL** — Secure Sockets Layer - vecākais drošu tīkla savienojumu protokols, kurš mūsdienās aizstāts ar drošāku TLS; termins joprojām plaši lietots kā sinonīms HTTPS savienojumiem [12]

**TLS** — Transport Layer Security - kriptogrāfisks protokols, kas nodrošina datu šifrēšanu un autentifikāciju tīkla saziņā; sistēmā izmantota versija TLS 1.2 vai jaunāka visiem HTTPS savienojumiem [12]

**TTL** — Time To Live - vērtība, kas nosaka, cik ilgi dati ir derīgi kešatmiņā vai tīklā; sistēmā izmantots Redis kešs un JWT refresh token derīguma termiņa kontrolei [9]

**TypeScript** — JavaScript valodas paplašinājums, ko izstrādājusi Microsoft, ar statisko tipizāciju un objektorientētas programmēšanas iespējām; sistēmā izmantots visā frontend kodabāzē, ievērojami samazinot izpildlaika kļūdas [2]

**Vite** — moderns frontend būvēšanas rīks ar iebūvētu HMR atbalstu un ESM bāzētu izstrādes serveri; sistēmā izmantots React/TypeScript projekta kompilēšanai un ātras izstrādes videi [2]

**WebSocket** — tīkla protokols, kas nodrošina pastāvīgu, divvirzienu saziņas kanālu starp klientu un serveri caur vienu TCP savienojumu; sistēmā pamats SignalR reāllaika komunikācijai un IoT datu straumēšanai [7]

**xUnit** — atvērta koda vienību testēšanas ietvars .NET platformai; sistēmā izmantots visu 73 automatizēto testu rakstīšanai, integrēts GitHub Actions CI/CD cauruļvadā [13]

**Backend** — servera puse (servera daļa) - sistēmas komponents, kas darbojas servera pusē un ir atbildīgs par biznesa loģikas aprēķiniem, datu bāzes pierēķināšanu un API endpointu nodrošināšanu; sistēmā realizēts ar ASP.NET Core (C#) [1]

**Frontend** — klienta puse (klienta daļa) - sistēmas komponents, kas darbojas lietotāja pārlūkprogrammā un ir atbildīgs par lietotāja saskarni un vizualizāciju; sistēmā realizēts ar React un TypeScript [2]

**ASP.NET Core** — Microsoft starpplatformu ietvars (framework) tīmekļa lietotņu un API izstrādei ar C# valodu; nodrošina augstu veiktspēju, iebūvētu DI konteineru, JWT autentifikāciju un SignalR integrāciju [1]

**PostgreSQL** — atvērta koda relāciju datubāzes pārvaldības sistēma ar pilnu ACID atbilstību; sistēmā galvenā datubāze strukturētu datu (lietotāji, dzīvnieki, adopcijas pieteikumi) glabāšanai [3]

**GitHub Actions** — GitHub platformas automatizācijas rīks CI/CD cauruļvadu konfigurēšanai YAML formātā; sistēmā automātiski palaiž testus, veic Docker attēlu būvēšanu un izvietojumu pēc katra koda iesnieguma [13]

**Dashboard** — informācijas pārskata panelis - lietotāja saskarnes elements, kas vienā skatā apkopo un vizualizē galvenos sistēmas rādītājus; sistēmā attēlo IoT sensoru reāllaika datus, brīdinājumus un adopciju statistiku

**Endpoint** — API galapunkts - konkrēts URL ceļš un HTTP metode, caur kuru klienti var veikt pieprasījumus backend servisiem; piemēram, POST /login vai GET /animals/{id}

**Middleware** — starpslānis - ASP.NET Core apstrādes cauruļvada komponents, kas izpilda kodu pirms vai pēc pieprasījuma apstrādes; sistēmā izmantots JWT validācijai, CORS politikai, pieprasījumu žurnālēšanai un kļūdu pārtveršanai [1]

## ANOTĀCIJA

Kvalifikācijas darbs veltīts programmatūras sistēmas “Smart Shelter IoT” izstrādei dzīvnieku patversmes pārvaldības procesu automatizācijai un apkārtējās vides parametru uzraudzībai. Darba mērķis ir radīt funkcionāli pilnvērtīgu, integrētu platformu, kas apvieno tīmekļa lietotni, servera daļu un lietu interneta (IoT) aparatūras komponentus.

Sistēmas izstrādē izmantotas šādas tehnoloģijas: frontend - React ar TypeScript; backend - ASP.NET Core (C#); datubāzes: PostgreSQL, MongoDB un Redis; IoT apakšsistēmai - ESP32 mikrokontrolleri ar CO<sub>2</sub>, temperatūras un mitruma sensoru; reāllaika komunikācijai - SignalR. Sistēma izvietota, izmantojot Docker Compose konteinerizāciju un Nginx apgriezta starpniekservera risinājumu.

Darba rezultātā izstrādāta funkcionāla programmatūras sistēma, kas nodrošina dzīvnieku uzskaiti, adoptācijas procesu pārvaldību, reāllaika mikroklimata monitoringu un lietotāju komunikāciju. Sistēmas kvalitāte apstiprināta ar automatizētu testēšanu (xUnit), ietverot vienību, integrācijas un API endpointu testus. Visi testi noslēgti veiksmīgi, kļūdu skaits — nulle.

## IEVADS

Mūsdienu straujās digitālo tehnoloģiju attīstības apstākļos un to aktīvās ieviešanas dažādās sabiedrības dzīves jomās īpaši aktuāla kļūst vadības un kontroles procesu automatizācija organizācijās, kas saistītas ar dzīvnieku aprūpi. Dzīvnieku patversmes pilda nozīmīgu sociālo funkciju, nodrošinot bezpajumtes dzīvnieku pagaidu uzturēšanu, ārstēšanu un jaunu īpašnieku meklēšanu. Tomēr vairumā gadījumu to darbība ir saistīta ar vairākām būtiskām problēmām, kas izriet no nepietiekamas digitalizācijas, centralizētu vadības sistēmu trūkuma un ierobežotām iespējām kontrolēt dzīvnieku uzturēšanas apstākļus.

Tradicionālās uzskaites metodes patversmēs bieži balstās uz papīra dokumentāciju vai sadrumstalotiem elektroniskiem rīkiem, kas noved pie datu zuduma, informācijas dublēšanās un apgrūtina piekļuvi aktuālajiem datiem. Vienotas informācijas sistēmas trūkums sarežģī dzīvnieku pārvaldības procesus, to stāvokļa kontroli, kā arī sadarbību starp darbiniekiem un potenciālajiem dzīvnieku īpašniekiem. Tas savukārt samazina patversmes darba efektivitāti un var negatīvi ietekmēt dzīvnieku labklājību.

Ne mazāk nozīmīga problēma ir mikroklimata kontrole telpās, kurās tiek turēti dzīvnieki. Apkārtējās vides parametri, piemēram, temperatūra, mitrums un oglekļa dioksīda (CO<sub>2</sub>) koncentrācija, tieši ietekmē dzīvnieku veselību, uzvedību un vispārējo stāvokli. Šo parametru neatbilstība noteiktajām normām var izraisīt slimību attīstību, paaugstinātu stresu un dzīves kvalitātes pasliktināšanos. Lielākajā daļā patversmju šo rādītāju kontrole tiek veikta neregulāri vai pilnībā ir atkarīga no cilvēka faktora, kas samazina tās uzticamību un precizitāti.

Mūsdienu informācijas tehnoloģiju sasniegumi, tostarp tīmekļa izstrāde, mākoņdatošana un lietu internets (IoT), rada priekšnoteikumus kompleksu risinājumu izstrādei, kas spēj apvienot pārvaldības, monitoringa un analīzes funkcijas vienotā sistēmā. IoT ierīču integrācija ļauj nepārtraukti iegūt datus par apkārtējās vides stāvokli reāllaikā, savukārt programmatūras sistēmas nodrošina šo datu apstrādi, vizualizāciju un izmantošanu pamatotu lēmumu pieņemšanai.

Šajā darbā tiek aplūkota programmatūras sistēmas “Smart Shelter IoT” izstrāde, kas paredzēta dzīvnieku patversmes pārvaldības procesu automatizācijai un pastāvīgai uzturēšanas apstākļu uzraudzībai. Piedāvātais risinājums ir integrēta platforma, kas ietver tīmekļa lietotni, servera daļu un IoT aparatūras komponentus, ļaujot vienotā sistēmā apvienot dzīvnieku uzskaiti, lietotāju pārvaldību, mikroklimata kontroli un mijiedarbību ar ārējiem lietotājiem.

Darba mērķis ir izstrādāt funkcionāli pilnvērtīgu programmatūras sistēmu, kas nodrošina efektīvu dzīvnieku patversmes pārvaldību un automatizētu apkārtējās vides parametru kontroli, izmantojot mūsdienu tehnoloģijas.

**Darba uzdevumi:**

- Izveidot platformas un datubāzes arhitektūru.
- Izstrādāt IoT sensoru mezglus CO<sub>2</sub>, temperatūrai mērījumiem un integrēt tos platformā.
- Realizēt arhitektūras komponentes dzīvnieku uzskaitēi, adopcijas procesa pārvaldībai un sensora datu apstrādei.
- Nodrošināt piekļuvi sistēmai, izmantojot DNS tuneli.
- Nodrošināt programmatūras automatizēto publicēšanu un testēšanu.

Darba praktiskā nozīme izpaužas iespējā ieviest izstrādāto sistēmu reālās dzīvnieku patversmēs, lai uzlabotu dzīvnieku aprūpes kvalitāti, optimizētu darba procesus un uzlabotu mijiedarbību ar lietotājiem. IoT tehnoloģiju izmantošana nodrošina nepārtrauktu uzturēšanas apstākļu kontroli, samazina cilvēka faktora ietekmi un ļauj savlaicīgi reaģēt uz novirzēm.

Tādējādi sistēmas “Smart Shelter IoT” izstrāde ir aktuāls un praktiski nozīmīgs uzdevums, kas vērsts uz dzīvnieku patversmju pārvaldības problēmu risināšanu, izmantojot mūsdienu informācijas tehnoloģijas. Projekta realizācija demonstrē teorētisko zināšanu pielietošanu programmēšanā un sistēmu projektēšanā, izstrādājot kompleksu programmatūras un aparatūras risinājumu.

Papildus praktiskajai nozīmei izstrādātā sistēma var tikt izmantota arī datu analīzes un zinātnisko pētījumu veikšanai. Uzkrātie dati par apkārtējās vides parametriem un dzīvnieku stāvokli ļauj veikt ilgtermiņa analīzi, identificēt likumsakarības un novērtēt dažādu faktoru ietekmi uz dzīvnieku veselību un labklājību. Šāda pieeja paver iespējas izmantot sistēmu ne tikai praktiskos nolūkos, bet arī kā instrumentu pētniecībā, piemēram, dzīvnieku uzvedības analīzē, mikroklimata optimizācijā un datu balstītu lēmumu pieņemšanā.

## 1. UZDEVUMA FORMULĒJUMS

Izstrādājamā programmatūras produkta nosaukums ir **“Smart Shelter IoT”**. Tā ir tīmekļa lietotne ar integrētu lietu interneta (IoT) sistēmu, kas paredzēta dzīvnieku patversmes pārvaldības procesu automatizācijai un apkārtējās vides parametru uzraudzībai reāllaikā.

Sistēmas galvenais mērķis ir nodrošināt centralizētu un efektīvu platformu, kas apvieno dzīvnieku uzskaiti, adopcijas procesu pārvaldību, lietotāju savstarpējo komunikāciju un mikroklimata monitoringu. Sistēma ir izstrādāta, lai uzlabotu dzīvnieku labklājību un optimizētu patversmes darbību.

### **Sistēmas pamata lietotāju grupas ir:**

Administrators – pārvalda sistēmu kopumā, lietotāju kontus, piekļuves tiesības un konfigurāciju;

Patversmes darbinieks – ievada un rediģē informāciju par dzīvniekiem, apstrādā adopcijas pieteikumus;

Veterinārārsts – piekļūst dzīvnieku medicīniskajiem datiem un analizē tos;

Klients (potenciālais adoptētājs) – apskata dzīvniekus, iesniedz adopcijas pieteikumus un sazinās ar patversmi.

### **Sistēmas realizācijai nepieciešami šādi galvenie elementi:**

- Frontend (tīmekļa lietotne) – nodrošina lietotāja saskarni (React);
- Backend (servera daļa) – realizē biznesa loģiku un API (ASP.NET Core);
- Datubāzes:
  - PostgreSQL – strukturētajiem datiem;
  - MongoDB – attēliem;
  - Redis – kešatmiņai;
- IoT apakšsistēma:
  - ESP32 mikrokontrolleri;
  - Sensori (CO<sub>2</sub>, temperatūra, mitrums);
- Reāllaika komunikācijas modulis (SignalR);

### **Sistēmas galvenie uzdevumi:**

- nodrošināt dzīvnieku uzskaiti un informācijas pārvaldību;
- organizēt adopcijas procesu (pieteikumu iesniegšana, apstiprināšana, komunikācija);
- nodrošināt apkārtējās vides parametru monitoringu reāllaikā;
- ģenerēt brīdinājumus kritisku rādītāju gadījumos;

- nodrošināt lietotāju savstarpējo komunikāciju (čats);
- uzkrāt datus un nodrošināt to analīzi;
- nodrošināt sistēmas paplašināmību, izmantojot AI/ML risinājumus.

**Sistēmas darbības nodrošināšanai nepieciešamās vides prasības:**

- operētājsistēmas: Windows, Linux vai macOS;
- tīmekļa pārlūkprogrammas: Google Chrome, Mozilla Firefox, Microsoft Edge;
- stabils interneta pieslēgums;
- servera vide ar Docker un Docker Compose atbalstu;
- IoT ierīces ar Wi-Fi savienojumu.

**Pieejamības nodrošināšana:**

Sistēma ir pieejama kā tīmekļa lietotne, kas darbojas pārlūkprogrammā bez nepieciešamības instalēt papildu programmatūru. Lietotāji var piekļūt sistēmai no jebkuras ierīces ar interneta pieslēgumu. IoT ierīces darbojas autonomi un nosūta datus uz serveri, izmantojot drošus sakaru protokolus.

## **2. PRASĪBU SPECIFIKĀCIJA**

### **2.1. Sistēmas funkcionālās prasības**

Šajā sadaļā specificētas galvenās sistēmas funkcijas, norādot ieejas datus, apstrādes loģiku un sagaidāmos rezultātus.

#### **2.1.1 Lietotāju reģistrācija un autentifikācija**

Lietotāju reģistrācija un autentifikācija ir pamata drošības mehānisms, kas ļauj lietotājiem izveidot kontus un piekļūt sistēmai ar atbilstošām tiesībām. Ieejas dati ietver e-pasta adresi, paroli, vārdu un lomu. Sistēma validē datus, šifrē paroli ar bcrypt [11], ģenerē JWT [10] tokenus un saglabā informāciju datubāzē. Rezultātā lietotājs saņem aktivizētu kontu un autentificētu sesiju ar RBAC (Role-Based Access Control) piekļuves līmeni.

- Ieejas dati: e-pasta adrese, parole, lietotāja vārds, izvēlētā loma.
- Apstrāde: sistēma validē datu formātu, pārbauda unikālu e-pasta eksistenci, šifrē paroli ar bcrypt algoritmu, ģenerē JWT tokenu un saglabā lietotāju datubāzē ar piešķirtajām tiesībām.
- Rezultāts: izveidots un aktivizēts lietotāja konts, nodrošināta autentificēta sesija ar atbilstošu piekļuves līmeni.

#### **2.1.2 Dzīvnieka profila izveide un rediģēšana**

Šī funkcija nodrošina pilnīgu dzīvnieku uzskaites pārvaldību patversmē. Ieejas dati ietver sugu, šķirni, vecumu, fotoattēlus un medicīnisko vēsturi. Dati tiek validēti, attēli augšupielādēti MongoDB [8], bet pamatinformācija saglabāta PostgreSQL. Rezultātā sistēmā tiek izveidots meklējams un filtrējams dzīvnieka profils.

- Ieejas dati: suga, šķirne, vecums, dzimums, fotoattēls(-i), medicīniskā vēsture, statuss (Pieejams/Adoptēts).
- Apstrāde: dati tiek validēti, fotoattēli optimizēti un augšupielādēti uz MongoDB, pamatinformācija strukturēti saglabāta PostgreSQL, piešķirts unikāls identifikators.
- Rezultāts: sistēmā izveidots vai atjaunots dzīvnieka profils, kas pieejams meklēšanai un filtrēšanai.

### **2.1.3. Adopcijas pieteikuma apstrāde**

Adopcijas pieteikuma apstrāde ir galvenais process, kas nodrošina pārskatāmu dzīvnieku nodošanu jauniem īpašniekiem. Ieejas dati ietver potenciālā adoptētāja identifikatoru, izvēlēto dzīvnieka ID, motivācijas aprakstu un dzīves apstākļu deklarāciju. Pieteikums tiek reģistrēts datubāzē ar statusu "Gaida izskatīšanu", darbinieks to pārskata, maina statusu un pievieno komentārus, bet sistēma automātiski ģenerē paziņojumu lietotājam. Rezultātā tiek izveidots pilnvērtīgs adopcijas process ar vēstures žurnālu un statusa maiņas notifikāciju.

- Ieejas dati: potenciālā adoptētāja identifikators, izvēlēto dzīvnieka ID, motivācijas apraksts, dzīves apstākļu deklarācija.
- Apstrāde: pieteikums reģistrēts datubāzē ar statusu "Gaida izskatīšanu", darbinieks to pārskata, pievieno komentārus un maina statusu uz "Apstiprināts" vai "Noraidīts". Sistēma automātiski ģenerē paziņojumu.
- Rezultāts: oficiāli reģistrēts adopcijas process ar pilnu vēstures žurnālu un statusa maiņas notifikāciju lietotājam.

### **2.1.4. Reāllaika čats starp adoptētāju un patversmi**

Reāllaika čats nodrošina ērtu un ātru komunikāciju starp potenciālajiem adoptētājiem un patversmes darbiniekiem. Ieejas dati satur ziņojuma tekstu, saistīto adopcijas vai dzīvnieka ID un sūtītāja identifikatoru. Ziņojums tiek momentāni pārraidīts, izmantojot SignalR [7] tehnoloģiju, un paralēli saglabāts PostgreSQL datubāzē ar laika zīmogu un kontekstu. Rezultātā lietotāji iegūst sinhronu komunikāciju ar pilnībā saglabātu vēsturi, kas saistīta ar konkrēto dzīvnieku vai pieteikumu.

- Ieejas dati: ziņojuma teksts, saistītā adopcijas ID vai dzīvnieka ID, sūtītāja ID.
- Apstrāde: ziņojums tiek šifrēts, izmantojot WebSocket/SignalR kanālu, momentāni pārraidīts saņēmējam, paralēli saglabāts PostgreSQL ar laika zīmogu un konteksta saitēm.
- Rezultāts: abpusēja, sinhrona komunikācija ar saglabātu vēsturi, kas saistīta ar konkrētu dzīvnieku vai pieteikumu.

### **2.1.5. IoT sensoru datu monitorings un brīdinājumu ģenerēšana**

IoT sensoru datu monitorings nodrošina nepārtrauktu apkārtējās vides parametru kontroli dzīvnieku telpās. Sistēmā nonāk telemetrija ( $\text{CO}_2$  ppm, temperatūra  $^{\circ}\text{C}$ , mitrums %) no ESP32 ierīcēm kopā ar ierīces ID un laika zīmogu. Dati tiek apstrādāti reāllaikā, vizualizēti dashboardā un salīdzināti ar normām, automātiski ģenerējot brīdinājumus personālam. Rezultātā patversmes darbinieki saņem aktuālu informāciju un savlaicīgus paziņojumus par iespējamajiem riskiem.

- Ieejas dati: sensoru telemetrija ( $\text{CO}_2$  ppm, temperatūra  $^{\circ}\text{C}$ , mitrums %) no ESP32 ierīcēm, ierīces ID, laika zīmogs.
- Rezultāts: reāllaika datu vizualizācija dashboardā, automātiski brīdinājumi personālam un notikumu žurnāls turpmākai analīzei

## **2.2 Sistēmas nefunkcionālās prasības**

Nefunkcionālās prasības nosaka sistēmas kvalitātes īpašības un darbības ierobežojumus.

### **2.2.1 Veiktspēja**

Veiktspējas prasības: IoT sensoru dati jāatjauno reāllaikā ar intervālu ne vairāk kā 60 sekunžu. Sistēmas atbildes laiks parastiem HTTP pieprasījumiem nedrīkst pārsniegt 1000 ms pie normālas slodzes. SignalR savienojumi jānodrošina reāllaikā ar latentumu zem 1000 ms.

### **2.2.2 Drošība**

Drošības prasības: visa datu pārsūtīšana jāveic caur HTTPS [12] (TLS 1.2 vai jaunāka versija). Autentifikācijai jāizmanto JWT [10] tokeni ar derīguma termiņu ne vairāk kā 15 minūtes; Refresh Token jāuzglabā HttpOnly cookie. Parolēm jātiek šifrētām ar bcrypt [11] algoritmu (sāļu skaits  $\geq 10$ ). Jārealizē piekļuves tiesību kontrole (RBAC) atbilstoši lietotāja lomai: Administrators, Patversme, Klients. Jānodrošina aizsardzība pret CSRF uzbrukumiem, kā arī pieprasījumu biežuma ierobežošana (Rate Limiting) caur Redis.

### **2.2.3 Lietojamība**

Lietojamības prasības: sistēmas saskarne jābūt reaģīvai (responsīvai) un jādarbojas korekti gan galddatoriem, gan viedtālruniem. Saskarnes valoda — angļu. Galvenās pamatfunkcijas (dzīvnieka meklēšana, adoptācijas pieteikums, sensoru skatu) jāspēj apgūt jauns lietotājs bez

papildu apmācības ne vairāk kā 10 minūšu laikā. Kritisko brīdinājumu paziņojumiem jābūt skaidri vizuāli atšķiramiem dashboardā.

#### **2.2.4 Saderība**

Sistēmai jābūt saderīgai ar noteiktiem klientu un izpildes vides nosacījumiem, lai nodrošinātu tās darbību gan web lietotājiem, gan servera komponentēm, gan IoT ierīcēm.

Pārlūkprogrammu saderība (frontend) - sistēmai jāspēj darboties šādās pārlūkprogrammās:

Google Chrome (versija  $\geq 110$ ), Mozilla Firefox (versija  $\geq 110$ ), Microsoft Edge (versija  $\geq 110$ ).

Servera vides saderība (backend un infrastruktūra) - Backend lietojumprogrammai un nepieciešamajām infrastruktūras komponentēm jāspēj darboties šādās operētājsistēmās:

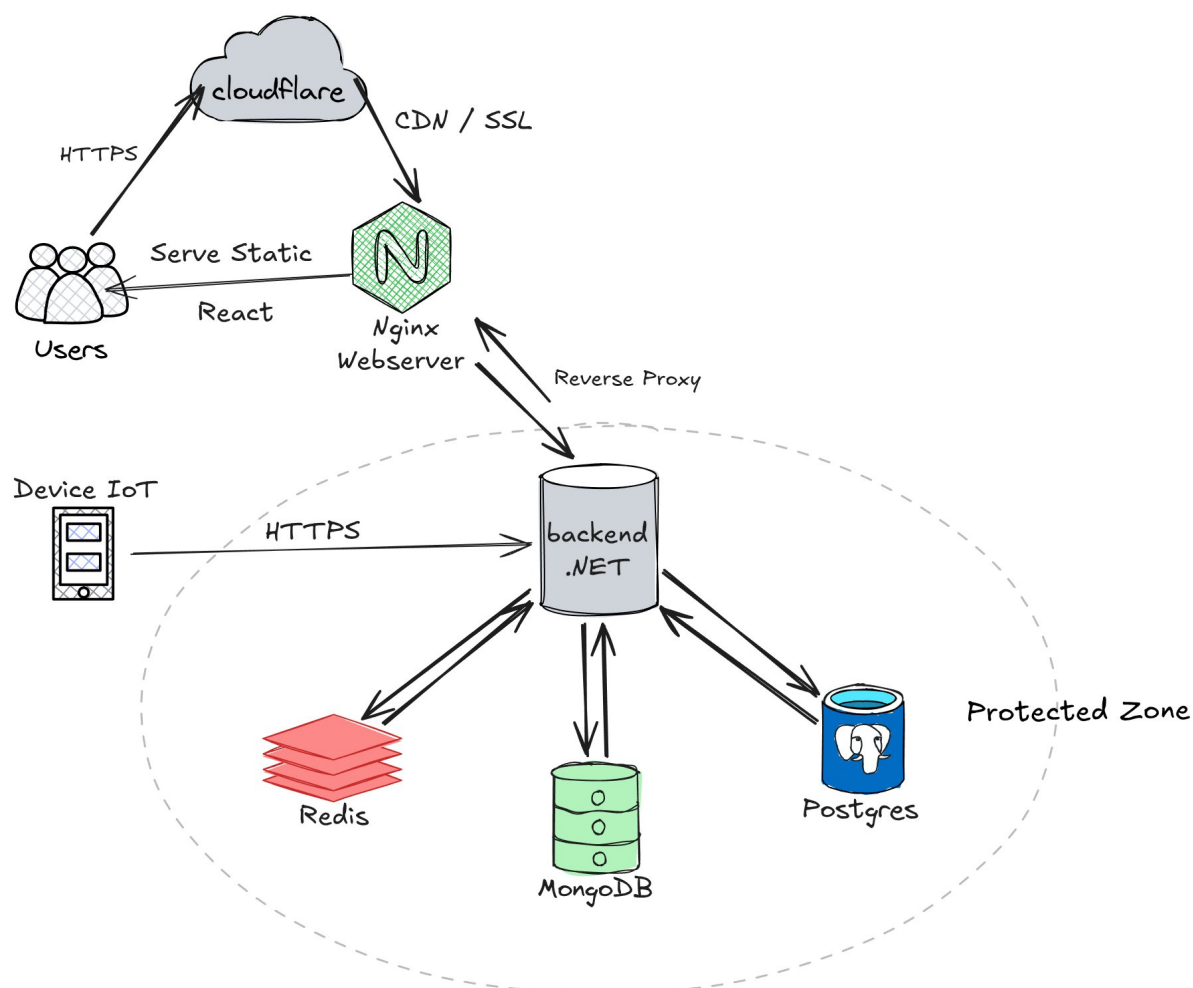
Linux (Ubuntu 22.04+), Windows Server 2019+, macOS 13+.

Sistēmas izvietošana ar Docker Compose nodrošina portabilitāti starp vidēm, vienoti definējot pakalpojumu konfigurāciju (backend, frontend, datubāzes un kešatmiņas komponentes).

IoT ierīču saderība - IoT ierīcēm (ESP32) jābūt savietojamām ar backend API komunikācijas prasībām: datu apmaiņai jāizmanto HTTPS protokols un komunikācijai jāatbilst backend API kontraktam (endpointi un autentifikācijas prasības).

### 3. Sistēmas tehniskais apraksts

#### 3.1 Sistēmas Arhitektura



3.1. attēls. "Smart Shelter IoT" sistēmas arhitektūras shēma (Alans Arzumanjans, 2026)

Sistēmas arhitektūra ir modulāra un daudzslāņu, kā redzams 3.1. attēlā. Tā sastāv no lietotāja slāņa (React frontend), perimetra aizsardzības (Cloudflare un Nginx), lietojumprogrammas slāņa (ASP.NET Core backend), reāllaika komunikācijas slāņa (SignalR un Redis), IoT komunikācijas slāņa (ESP32 ierīces ar HTTPS) un datu glabāšanas slāņa (PostgreSQL, MongoDB un Redis). Šī arhitektūra nodrošina skaidru atbildību sadali starp komponentēm, augstu drošību un iespēju sistēmu mērogot.

Izvēloties galvenās tehnoloģijas, tika rūpīgi izvērtētas alternatīvas, ņemot vērā projekta prasības pēc veiktspējas, drošības, reāllaika datu apstrādes un izstrādes efektivitātes.

Backend izstrādei tika izvēlēts ASP.NET Core [1] (C#). Šī tehnoloģija nodrošina augstu veiktspēju, iebūvētus drošības mehānismus (JWT, CORS, Rate Limiting), ērtu SignalR integrāciju reāllaika komunikācijai un lielisku atbalstu Entity Framework Core. Salīdzinājumā ar Python (Django vai FastAPI) C# ir ātrāks reāllaika IoT datu apstrādē, jo Python Globālais interpretatora bloķēšanas mehānisms (GIL) ierobežo paralēlo izpildi. Node.js (Express.js) savukārt prasa vairāk manuālas konfigurācijas drošībai un transakcijām, kas palielina kļūdu risku kompleksā sistēmā.

Frontend daļai izvēlēts React [2] ar TypeScript. Tas nodrošina ātru, komponentu balstītu saskarni, virtuālo DOM un statisko tipu pārbaudi, kas ievērojami samazina kļūdas izstrādes laikā. Angular ir pārāk smags un sarežģīts šāda mēroga projektam, bet Vue.js ir vieglāks, taču tam ir mazāka ekosistēma un mazāk gatavu risinājumu sarežģītiem dashboardiem un reāllaika vizualizācijai.

Datu glabāšanai izmantota poliglota pieeja: PostgreSQL [3] strukturētajiem datiem (lietotāji, dzīvnieki, adopcijas), MongoDB attēliem un dokumentiem, kā arī Redis [9] kešatmiņai un reāllaika operācijām. PostgreSQL izvēlēts tāpēc, ka tas nodrošina pilnu ACID atbilstību un spēcīgu JSON atbalstu. MySQL ir vienkāršāks, bet tam ir vājāki rīki sarežģītiem vaicājumiem un paralēlām transakcijām. Firebase piedāvā ērtu reāllaika sinhronizāciju, taču rada piegādātāja piesaistes risku un augstas izmaksas pie lielāka datu apjoma.

IoT komunikācijai izmantoti ESP32 [4] mikrokontrolleri ar HTTPS protokolu. Šī kombinācija ir izmaksu efektīva, energoefektīva un droša. Reāllaika komunikācijai SignalR ar Redis backplane izvēlēts tāpēc, ka tas ļauj viegli mērogot un nodrošina stabilu WebSocket savienojumu pārvaldību, ko ir sarežģītāk realizēt ar citiem risinājumiem. Kopumā izvēlētās tehnoloģijas nodrošina optimālu līdzsvaru starp veiktspēju, drošību un izstrādes ātrumu, ļaujot izveidot stabilu, mērogojamu un uzturējamu sistēmu dzīvnieku patversmes pārvaldībai.

### **3.2. Līdzekļu Tehnoloģiju un valodu apraksts**

Projekta „Smart Shelter IoT” realizācijai tika izvēlēta modulāra, uz veiktspēju un drošību orientēta tehnoloģiju kaudze, kas nodrošina efektīvu datu apstrādi, reāllaika monitoringu un mērogojamu infrastruktūru. Zemāk detalizēti aprakstītas izmantotās programmēšanas valodas, ietvari un platformas, kā arī pamatojums to izvēlei.

## **Programmēšanas valodas un ietvari**

- C# ar ASP.NET Core 8: Backend loģikas izstrādei izmantota C# valoda kopā ar ASP.NET Core 8 ietvaru. Tā nodrošina augstu veiktspēju, statisko tipu drošību, integrētu atkarību injekciju (DI) konteineru un pilnvērtīgu asinhronās programmēšanas atbalstu. Datu bāzes mijiedarbībai pielietots Entity Framework Core, kas automatizē migrācijas un nodrošina tipdrošus LINQ vaicājumus.
- TypeScript ar React un Vite: Frontend izstrādei izvēlēta TypeScript valoda, kas pievieno JavaScript statisko tipizāciju, būtiski samazinot izpildlaika kļūdas un uzlabojot koda uzturamību. React bibliotēka ar Vite bundleru nodrošina ātru karsto atjaunināšanu (HMR), efektīvu virtuālā DOM renderēšanu un komponentu modulāru arhitektūru. Interfeisa stilizācijai izmantots TailwindCSS, kas ļauj strauji izveidot responsīvu un vizuāli konsekventu lietotāja pieredzi.
- C++ (Arduino ekosistēma): IoT mezglu (ESP32D) firmware izstrādei izmantota C++ valoda. Tā nodrošina tiešu piekļuvi mikrokontroliera aparatūras reģistriem, minimālu atmiņas patēriņu un deterministisku izpildes laiku, kas ir obligāts nosacījums sensoru datu precīzai un nepārtrauktai vākšanai.

## **Datu glabāšana un reāllaika komunikācija**

- PostgreSQL: Relāciju datubāze izmantota strukturētu biznesa datu glabāšanai (lietotāju konti, dzīvnieku profili, adopcijas pieteikumi, pasūtījumi). Tā garantē pilnu ACID atbilstību, nodrošina sarežģītu relāciju un transakciju pārvaldību, kā arī piedāvā iebūvētu JSON un ģeotelpisko datu atbalstu.
- MongoDB [8]: Dokumentu datubāze izvēlēta IoT telemetrijas žurnālu un attēlu materiālu glabāšanai. Tās bezshēmas (schema-less) uzbūve un optimizēta rakstīšanas caurlaidība ir ideāli piemērotas augstas intensitātes laika rindu (time-series) datiem.
- Redis [9]: Atmiņā balstīta (in-memory) datu struktūru krātuve kalpo kā kešatmiņa, reāllaika ierīču stāvokļa glabātuvē, SignalR backplane sinhronizācijas slānis un API ātruma ierobežošanas (rate limiting) mehānisms. Tā nodrošina atbildes laikus mikrosekunžu diapazonā.
- SignalR & WebSockets: Nodrošina divvirzienu, zemas latentuma sakaru kanālu starp klientu pārlūkprogrammām un serveri, kas ir kritiski IoT dashboarda atjaunināšanai un integrētās tērzesšanas funkcionalitātei.

## Infrastruktūra un DevOps

- Docker & Docker Compose: Visa sistēma konteinerizēta, nodrošinot vides izolāciju, reproducējamību un vieglu horizontālu mērogojamību.
- Nginx & Cloudflare: Nginx [14] darbojas kā apgrieztā starpniekservera (reverse proxy) un statisko React failu pasniešanas serviss. Cloudflare nodrošina perimetra drošību (WAF), DDoS aizsardzību, SSL/TLS termināciju un globālo CDN piegādi.
- GitHub Actions [13]: Automatizēts CI/CD cauruļvads, kas nodrošina nepārtrauktu kodu būvēšanu, automatizētu testu palaišanu un konteineru izvietošanu serverī.

## Izvēles pamatojums un salīdzinājums ar alternatīvām

Projektēšanas posmā tika izvērtētas vairākas tehnoloģiskās alternatīvas. Backend izstrādei tika apsvērti Node.js (NestJS/Express), Python (FastAPI/Django) un Java (Spring Boot). C# ar ASP.NET Core 8 tika izvēlēts, jo tas demonstrē augstāko veiktspēju sintētiskos un reālos slodzes testos, piedāvā vislabāko tipu drošību, iebūvētu identitātes un drošības pārvaldību (JWT, Role-Based Access Control), kā arī vienotu ekosistēmu, kas atvieglo IoT datu plūsmu un reāllaika apstrādi.

Frontend alternatīvas ietvēra Angular, Vue.js un Svelte. React kopā ar TypeScript tika izvēlēts lielākās izstrādātāju kopienas, plašā trešo pušu bibliotēku ekosistēmas un Vite piedāvātās ātrās izstrādes vides dēļ, kas ir īpaši svarīgi iteratīvai projektēšanai un ātrai prototipēšanai.

Datu bāzu ziņā tika vērtētas MySQL (kā PostgreSQL alternatīva), SQLite (viegls variants), kā arī specializētās laika rindu DB InfluxDB vai TimescaleDB. PostgreSQL tika izvēlēts tā robustās transakciju garantiju, uzlaboto JSON apstrādes iespēju un zemas uzturēšanas prasību dēļ, savukārt MongoDB nodrošina nepieciešamo elastību un rakstīšanas caurlaidību telemetrijas datiem, ko tradicionālās relāciju datubāzes nevarētu nodrošināt tik efektīvi bez sarežģītas shēmu pārvaldības.

Kopumā izvēlētie risinājumi atšķiras no alternatīvām ar labāku savstarpējo integrāciju, augstāku veiktspēju reāllaika un IoT scenārijos, skaidrāku datu slāņu atdalīšanu (strukturēti biznesa dati pret nestrukturētiem sensoru datiem), kā arī modernu DevOps atbalstu, kas nodrošina ātru, drošu un viegli uzturamu programmatūras dzīves ciklu.

### 3.3 IoT apakšsistēma

IoT apakšsistēma ir būtiska sistēmas daļa, kas nodrošina apkārtējās vides parametru monitoringu. Tā sastāv no: ESP32D mikrokontrolleriem un Sensirion SCD41 [6] sensoriem (CO<sub>2</sub>, temperatūra, mitrums). IoT ierīces vāc datus reāllaikā, nosūta tos uz serveri, izmantojot HTTPS [12], darbojas autonomi. Šie dati tiek izmantoti mikroklimata kontrolei, brīdinājumu ģenerēšanai, analīzei un prognozēšanai.

### 3.4 Datu plūsma sistēmā

Sistēmas datu plūsma ir projektēta kā asinhrons, daudzslāņu process, kas nodrošina efektīvu lietotāju pieprasījumu apstrādi un reāllaika IoT telemetrijas uztveršanu. Datu aprites loģiku var iedalīt četros galvenajos posmos (skatīt 3.1.attēls. *"Smart Shelter IoT" sistēmas arhitektūras shēma*):

#### 3.4.1. Statiskā satura ielāde un sākotnējā autentifikācija

Lietotājam atverot sistēmu pārlūkprogrammā, Nginx apgrieztais starpniekserveris nodrošina React frontend statisko failu (JavaScript, CSS, attēli) piegādi. Pēc veiksmīgas pieteikšanās backend ģenerē JWT un Refresh Token pārus, kurus droši uzglabā pārlūkprogrammā.

#### 3.4.2. CRUD pieprasījumu apstrāde

Veicot darbības ar dzīvnieku profiliem, adopcijas pieteikumiem vai lietotāju kontiem, lietotāja pārlūks ģenerē HTTP GET/POST/PUT/DELETE pieprasījumus. Tie šķērso Cloudflare drošības slāni (WAF un DDoS filtrēšana), nonāk Nginx, kas tos maršrutē uz ASP.NET Core backend. Backend validē JWT, pārbauda RBAC tiesības, veic biznesa loģikas apstrādi un mijiedarbojas ar datu glabāšanas slāni: strukturētie dati un IoT vēsture tiek lasīti/rakstīti PostgreSQL, attēli uzglabāti MongoDB, bet bieži pieprasītie vai sesijas dati tiek kešoti Redis.

#### 3.4.3. IoT telemetrijas uzņemšana

ESP32D mikrokontrolleri periodiski (pēc konfigurējama intervāla) vāc sensoru datus (CO<sub>2</sub>, temperatūra, mitrums) un nosūta tos backend API caur šifrētu HTTPS savienojumu, izmantojot API atslēgu autentifikāciju. Saņemtie mērījumi tiek momentāni validēti, normalizēti un saglabāti PostgreSQL kā laika rindu (time-series) žurnāls.

#### 3.4.4. Atbilšu formēšana un cikla noslēgums

Pēc datu apstrādes backend atgriež JSON atbildi ar statusa kodu (piem., 200 OK, 201 Created, 400 Bad Request). Nginx saņem atbildi un nodrošina tās piegādi lietotāja pārlūkprogrammai. React frontend reaktīvi atjauno DOM, attēlojot datus, veidlapas vai kļūdu paziņojumus. Šis cikls atkārtojas katram lietotāja un IoT ierīces pieprasījumam, nodrošinot zemu latentumu, augstu caurlaidību un datu integritāti visos slāņos.

### 3.5 Drošības arhitektūra

Sistēmā tiek īstenota daudzslāņu (Defense-in-Depth) drošības pieeja, kurā aizsardzība tiek nodrošināta vairākos līmeņos: perimetrā, tīkla/ieviešanas līmenī, lietojumprogrammas loģikas līmenī un datu aizsardzības līmenī.

#### 3.5.1) Perimetra drošība (Cloudflare)

Visa ārējā satiksme vispirms nonāk pie Cloudflare - mākoņpakalpojuma, kas darbojas kā "digitālais vārtsargs":

Filtrē ļaunprātīgu trafiku - bloķē masveida uzbrukumus (DDoS), kas mēģina "noslīcināt" serveri;

Web Application Firewall (WAF) - automātiski atpazīst un bloķē biežākos tīmekļa uzbrukumus (piemēram, mēģinājumus ielauzties datubāzē);

Šifrē savienojumu - visi dati starp lietotāju un sistēmu tiek pārsūtīti šifrētā veidā (HTTPS), lai tos nevarētu nolasīt trešās personas.

Rezultāts: Uzbrucējs neredz iekšējās infrastruktūras detaļas, un tiešie uzbrukumi serverim tiek būtiski samazināti.

#### 3.5.2) Tīkla un infrastruktūras izolācija (Docker sastāvs)

Sistēmas komponentes darbojas atsevišķos, izolētos konteineros:

Datubāzes (PostgreSQL, MongoDB, Redis) nav pieejamas no interneta — tām var piekļūt tikai pašas sistēmas iekšējie komponenti;

Frontend (lietotāja saskarne) un backend (servera loģika) sazinās caur iekšējo tīklu;

Ārējiem lietotājiem ir pieejami tikai divi porti: 80 (HTTP) un 443 (HTTPS).

*Analoģija:* Iedomājieties, ka datubāzes atrodas slēgtā telpā, kurā var iekļūt tikai ar īpašu atslēgu, un šī atslēga ir tikai servera loģikas rīcībā.

### 3.5.3) Lietojumprogrammas drošība

Pieteikšanās: Lietotājs ievada e-pastu un paroli. Sistēma pārbauda datus un izsniedz divus "marķierus":

- *Īslaicīgs marķieris (15 minūtes) - ļauj veikt darbības sistēmā;*
- *Atjaunošanas marķieris (ilgāks termiņš) - ļauj saņemt jaunu īslaicīgo marķieri, nepiesakoties no jauna.*

Droša glabāšana: Atjaunošanas marķieris tiek saglabāts pārlūkprogrammā tā, lai to nevarētu nolasīt ļaunprātīgs JavaScript kods.

Lomu pārvaldība: Katram lietotājam ir piešķirta loma (Administrators, Patversme, Klients). Sistēma automātiski pārbauda, vai lietotājam ir tiesības veikt konkrētu darbību.

### 3.5.4) Aizsardzība pret biežākajiem uzbrukumiem

| Uzbrukuma veids                                                  | Kā sistēma aizsargājas                                                                   |
|------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| Brute-force (daudz mēģinājumu uzminēt paroli)                    | Ierobežo pieteikšanās mēģinājumus no vienas IP adreses (piem., 10 mēģinājumi 15 minūtēs) |
| CSRF (ļaunprātīgas vietnes mēģina izmantot lietotāja sesiju)     | Cookie bāzētām darbībām tiek izmantots antiforgery/CSRF mehānisms                        |
| XSS (ļaunprātīga skriptu ievietošana)                            | Nginx serveris nosūta drošības galvenes, kas bloķē šādu skriptu izpildi                  |
| Clickjacking (lietotājs tiek "apmānīts" noklikšķināt uz kaut kā) | CSP politika neļauj ielādēt sistēmu ietvaros (iframe) citās vietnēs                      |

CORS konfigurācija ļauj pieprasījumus tikai noteiktiem frontend izcelsmes domēniem, kas tiek nolasīti no vides mainīgajiem. Tiek izmantots iestatījums, kas ir saderīgs ar cookie bāzētu refresh token shēmu.

### 3.5.5) Pieprasījumu ierobežošana (rate limiting)

Sensitīvu datu šifrēšana: Lietotāju e-pasti un tālruņi pirms saglabāšanas tiek šifrēti. Pat ja kāds iegūst piekļuvi datubāzei, šos datus nevarēs nolasīt bez atslēgas.

Paroļu aizsardzība: Paroles netiek glabātas kā parasts teksts. Tās tiek pārveidotas par unikālu "parakstu" (hash), ko nevar atgriezt atpakaļ. Pat ja datubāze tiek nopludināta, paroles paliek drošas.

Sesiju kontrole: Logout un refresh laikā refresh token jti tiek anulēts Redisā.

### **3.5.6) Datu drošība**

Reverse proxy serveris Nginx pievieno papildu aizsardzības slāni:

- Drošības galvenes: Informē pārlūkprogrammu, kā droši apstrādāt saturu (piemēram, neļaut ielādēt resursus no neuzticamiem avotiem);
- Kompresija un kešēšana: Paātrina ielādi, vienlaikus saglabājot drošību;
- SignalR atbalsts: Nodrošina drošu reāllaika komunikāciju (čats, sensoru dati).

Sistēmas drošības arhitektūra balstās uz Defense-in-Depth principu un sastāv no perimetra aizsardzības (Cloudflare), infrastruktūras izolācijas (Docker Compose), drošas autentifikācijas un autorizācijas (JWT + refresh cookie + Redis revocation), kā arī papildus aizsardzības mehānismiem (CSRF, CORS, rate limiting) un sensitīvo datu šifrēšanas. Nginx konfigurācija papildus ievieš drošības galvenes (t.sk. CSP), samazinot XSS un citu web slāņa risku iespējamību

## **3.6 Sistēmas mērogojamība un paplašināmība**

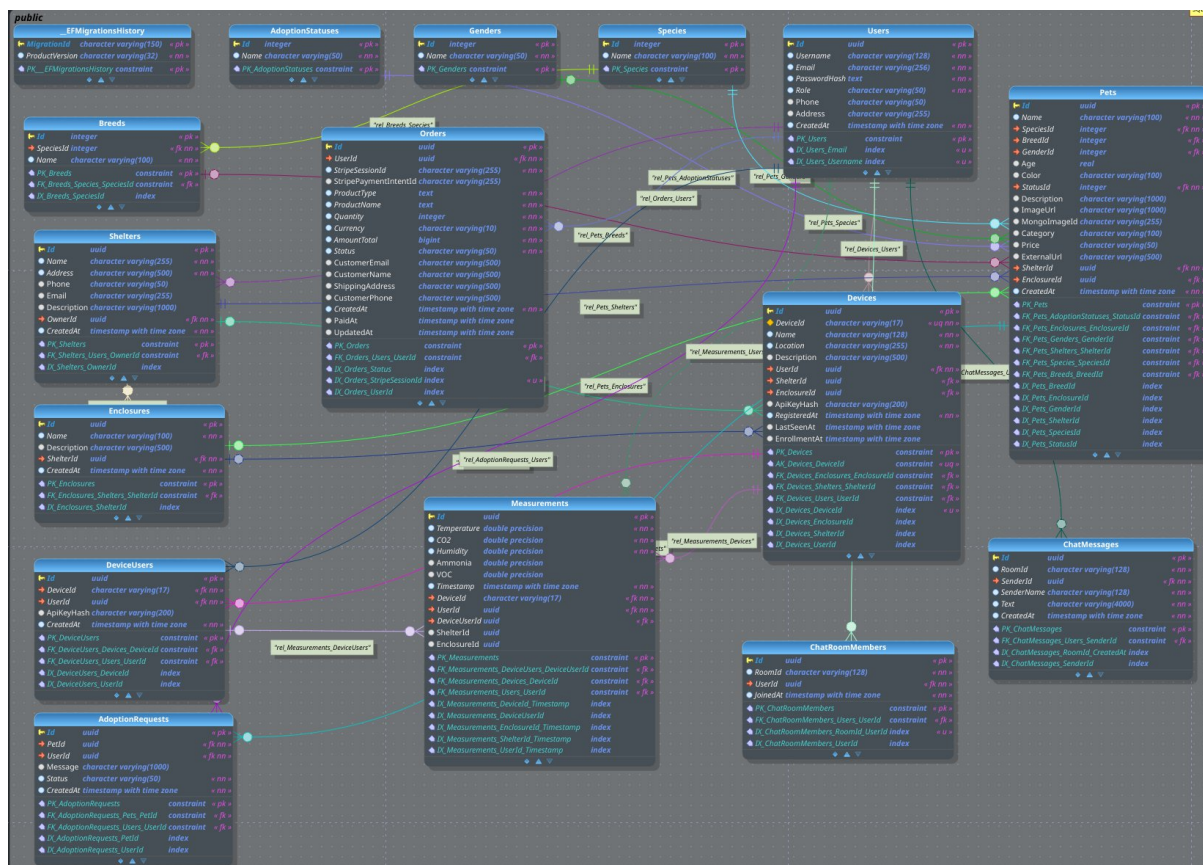
Sistēma ir izstrādāta ar iespēju to paplašināt:

- iespējams pievienot jaunus IoT sensorus;
- iespējams integrēt AI/ML moduļus;
- iespējams palielināt lietotāju skaitu bez būtiskām izmaiņām arhitektūrā;
- iespējams sadalīt sistēmu mikroservisos nākotnē.

Sistēmas arhitektūra un konteineru tehnoloģiju izmantošana nodrošina vieglu jaunu komponentu pievienošanu un ātru izvietošanu dažādās vidēs. Ar Docker Compose un Nginx palīdzību sistēmu var mērogot horizontāli, pievienojot papildu servisa instancēs, nemainot kodolu arhitektūru.

### 3.7 Datubāzes struktūra un ER modelis

Sistēmas datu glabāšanai tika izmantota poliglota pieeja, kur PostgreSQL nodrošina strukturēto datu glabāšanu, savukārt MongoDB un Redis tiek izmantoti specifiskiem uzdevumiem. Galveno datubāzes entītiju struktūra un savstarpējās attiecības ir attēlotas 3.7. attēlā.



### 3.7. attēls. Datubāzes ER modelis (Alans Arzumanjans, 2026)

Izstrādātā ER diagramma atspoguļo sistēmas galveno objektu struktūru un to savstarpējo saistību modeli. Datu modelis veidots, ievērojot normalizācijas principus un nodrošinot datu integritāti, izmantojot primārās (PK) un ārējās atslēgas (FK).

Centrālā sistēmas entītija ir `Users`, kas glabā lietotāju informāciju un ir saistīta ar vairākām citām sistēmas komponentēm. Lietotāji var pārvaldīt patversmes (`Shelters`), veidot adopcijas pieprasījumus (`AdoptionRequests`), izmantot ierīces un piedalīties komunikācijā.

Dzīvnieku dati tiek glabāti entītijā `Pets`, kas satur informāciju par dzīvnieku raksturojumu, statusu, vecumu, aprakstu un atrašanās vietu. Papildus izmantotas palīgēntītijas `Species`, `Breeds`, `Genders` un `AdoptionStatuses`, kas samazina datu dublēšanos un nodrošina konsekventu klasifikāciju.

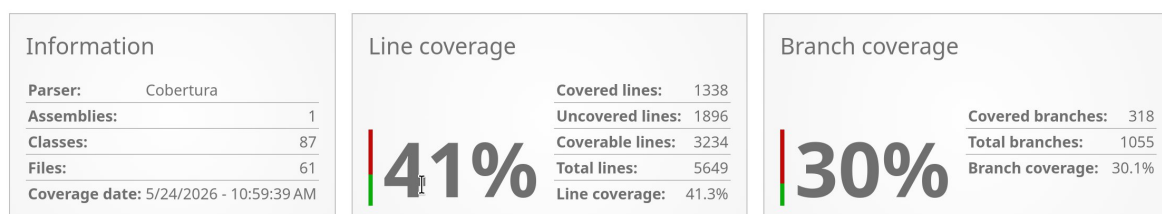
IoT apakšsistēma realizēta, izmantojot entītijas `Devices` un `Measurements`. Ierīces tiek saistītas ar konkrētām patversmēm un voljēriem, savukārt mērījumu tabula uzkrāj telemetrijas datus, piemēram, temperatūru, CO<sub>2</sub> koncentrāciju, mitrumu, amonjaka un gaistošo organisko savienojumu rādītājus.

Komunikācijas funkcionalitāte realizēta ar `ChatMessages` un `ChatRoomMembers` entītijām, kas nodrošina reāllaika ziņojumu apmaiņu starp lietotājiem.

Datu modelis izstrādāts ar iespēju to paplašināt nākotnē, pievienojot jaunus sensorus, moduļus un papildu sistēmas komponentes bez būtiskām arhitektūras izmaiņām.

## 4. Programmatūras testēšana un rezultāti

Programmatūras testēšana ir neatņemama izstrādes dzīves cikla sastāvdaļa, kas nodrošina sistēmas funkcionalitātes, stabilitātes un drošības verifikāciju pirms tās nodošanas ekspluatācijā. Šajā nodaļā aprakstīta izstrādātās sistēmas testēšanas stratēģija, izmantotā metodoloģija un empīriskie rezultāti. Testēšanas process tika organizēts saskaņā ar modernās programmatūras inženierijas labākās prakses standartiem, ietverot izolētus vienību testus, integrācijas testus ar imitētām infrastruktūras atkarībām, kā arī API līmeņa funkcionalitātes pārbaudes. Visi testi tika automatizēti un integrēti CI/CD cauruļvadā, nodrošinot katra koda izmaiņu validāciju pirms iekļaušanas produkcijas versijā. Turpmākajā apakšnodaļās detalizēti aplūkoti testu pārklājuma rādītāji, izmantotie instrumenti, testēšanas vides konfigurācija, kā arī sistēmas veiktspējas un drošības validācijas rezultāti.



### 4.1. attēls. Testu pārklājuma pārskats (ReportGenerator Cobertura, 2026)

Saskaņā ar kvalifikācijas eksāmena noteikumiem (3.e. punkts), izstrādātā risinājuma kvalitātes nodrošināšanai tika veikta sistemātiska programmatūras testēšana, iekļaujot vienību, integrācijas un funkcionālo testu kopas. Testēšanas process tika automatizēts un integrēts CI/CD cauruļvadā, nodrošinot katra koda izmaiņu validāciju pirms iekļaušanas produkta versijā.

4.1. attēls attēlo testu pārklājuma pārskatu, kas ģenerēts ar ReportGenerator rīku, izmantojot Cobertura pārklājuma datu formātu. Attēlā redzami trīs galvenie informācijas bloki, kas sniedz visaptverošu priekšstatu par izstrādātās sistēmas testēšanas kvalitāti.

Informācijas panelī norādīti vispārējie statistikas dati: analizēts 1 assembly, 87 klases un 61 fails. Pārskats ģenerēts 2026. gada 24. maijā.

Line coverage panelī redzams, ka kopējais rindu pārklājums ir 41,3 %. No 5649 kopējām koda rindām pārbaudītas ir 1338 rindas, bet nepārbaudītas palikušas 1896 rindas. Šis rādītājs parāda, cik liela daļa no programmas koda ir izpildīta testa laikā.

Branch coverage panelī redzams, ka zaru pārklājums ir 30,1 %. No 1055 kopējiem zariem (loģiskajām izvēlēm if, else, ciklos u.c.) pārbaudīti ir 318 zari.

Galveno jēdzienu skaidrojums:

- Line Coverage (Rindu pārklājums) — rāda, cik procentuāli no programmas kodu rindām tika izpildītas testa izpildes laikā. Tas ir viens no pamata testu pārklājuma rādītājiem.
- Branch Coverage (Zaru pārklājums) — detalizētāks rādītājs, kas novērtē, cik dažādu loģisko zaru (ceļu) koda iekšienē tika izpildīti. Tas ir svarīgāks par rindu pārklājumu, jo parāda, vai testi aptver dažādus nosacījumus (if/else, null pārbaudes, kļūdu apstrādi u.c.).
- Cobertura — atvērta pirmkoda formāts un rīks testu pārklājuma datu vākšanai un analīzei.
- ReportGenerator — rīks, kas pārveido neapstrādātus pārklājuma datus (Cobertura) skaistos, cilvēkam saprotamos HTML pārskatos.

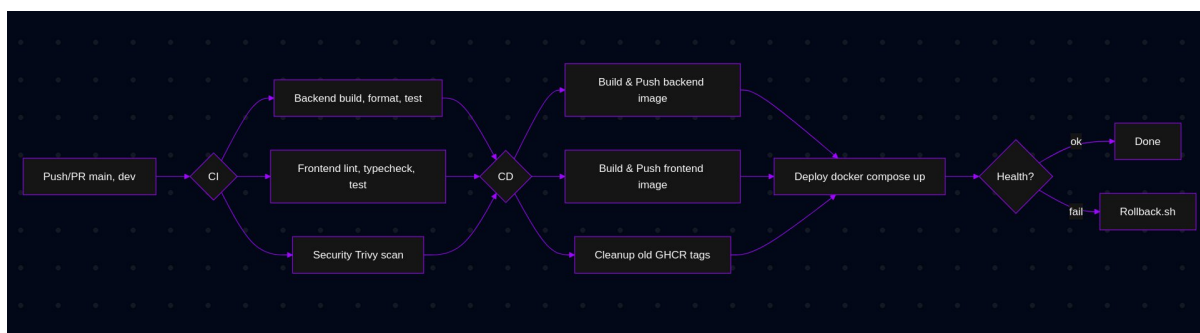
Neraugoties uz vidējo pārklājuma līmeni, visi 73 izstrādātie testi tika izpildīti sekmīgi (0 kļūdu). Augstākais pārklājums sasniegts kritiskajos komponentos (JwtService - 100 %, Services.UserEmailService - 88.8%).

#### **4.1. Testēšanas metodoloģija un pieeja**

Projekta izstrādē tika pielietota strukturēta testēšanas stratēģija, kurā testi tiek rakstīti paralēli koda izstrādei, lai nodrošinātu agrīnu kļūdu atklāšanu. Tika nošķirti šādi testēšanas līmeņi:

- Vienību testi (Unit Tests) – atsevišķu servisu, palīgklasju un algoritmu izolēta pārbaude bez ārējām atkarībām.
- Integrācijas testi (Integration Tests) – API slāņa, datubāzu mijiedarbības un autentifikācijas plūsmu validācija, izmantojot izolētu InMemory datubāzi un pielāgotus servisa dubultniekus.
- Endpointu testi – HTTPS pieprasījumu/atbilžu ciklu pārbaude, tostarp statusa kodu, atgriežamo datu struktūras un kļūdu apstrādes validācija.

## 4.2. Izmantotie rīki un testēšanas vide



4.2. attēls. Automatizētā CI/CD testēšanas cauruļvada shēma (Alans Arzumanjans, 2026)

Testēšanas ietvars: xUnit, Microsoft.AspNetCore.Mvc.Testing (WebApplicationFactory).

EF Core InMemory datubāzi, FakeRedisService kā redis interfeisa dubultnieku.

EF Core InMemory (datu slāņa izolācijai). Integrācijas testos API tiek palaista testu režīmā (TestServer), un visas datu operācijas tiek novirzītas uz EF Core InMemory provideri.

InMemory tiek izmantots, lai testu izpilde būtu ātra un neatkarīga no ārējām infrastruktūrām (piem., PostgreSQL servera). Tas ļauj testēt API kontrolleru un biznesa loģiku (CRUD darbības, validāciju, attiecību saglabāšanu) bez tīkla/DB pieejamības problēmām.

Lai nepieļautu datu savstarpēju piesārņošanu starp dažādiem testiem, InMemory datubāzes nosaukums tiek veidots unikāls katram testa servera izveides gadījumam (tests-{Guid}). Tādējādi katrs tests saņem savu “sandbox” vidi un iepriekšējā testa dati neietekmē nākamās rezultātus.

Pirms testiem nepieciešamie bāzes entītiņu dati (piem., Species, Breeds, Genders, AdoptionStatuses) tiek seedoti vienreiz test servera inicializācijas laikā. Papildus tam konkrētos testos dati tiek “sagatavoti” individuāli ar seed metodēm (SeedUserAsync, SeedShelterAsync, SeedPetAsync, SeedDeviceAsync), kas tieši raksta InMemory db.

Pēc HTTP pieprasījumu izpildes testos tiek pārbaudīts DB stāvoklis. To nodrošina Factory helperis ExecuteDbContextAsync, kas no izveidotās aplikācijas DI konteksta izņem svaigu ApplicationDbContext (izmantojot jaunu scope). Tas ļauj korekti validēt, ka API tiešām ir noturējusi izmaiņas datubāzē, ne tikai atgriežusi pareizu HTTP atbildi.

Par FakeRedisService (kešatmiņas un rate limiting izolācijai) - Aplikācijā redis tiek izmantots tādām funkcijām kā kešatmiņa un rate limiting, kā arī refresh token revocation. Integrācijas testos ārējs redis būtu nevajadzīga atkarība: tas palēninātu testus, radītu potenciālu flakiness (sinhronizācijas/timeout problēmas) un pieprasītu papildus infrastruktūras uzstādīšanu. Tāpēc

tiek ieviests FakeRedisService, kas implementē to pašu IRedisService interfeisu, bet datu glabāšanai izmanto lokālas atmiņas struktūras (ConcurrentDictionary un ConcurrentQueue). Kešatmiņas/glabāšanas darbībām FakeRedisService izmanto entries ar TTL (ExpiresAt). SetAsync saglabā vērtību (serializētu JSON) kopā ar derīguma termiņu, bet GetAsync atgriež vērtību tikai kamēr tā nav “expired”. Ja expired, ieraksts tiek izņemts. Rate limiting testēšanai FakeRedisService ievieš “rolling window” loģiku: katram rate limit taustiņam tiek uzturēta rindu struktūra ar pieprasījumu timestampiem, pirms atļaut pieprasījumu vecie ieraksti tiek iztīrīti, ja tie ir ārpus noteiktā laika loga (window), pieļaujamība tiek aprēķināta pēc queue izmēra pret limitu.

Refresh token atcelšanai (revocation) FakeRedisService izmanto atsevišķus ierakstus ar atslēgu formātu revoked\_rt:{jti}. IsRefreshTokenRevokedAsync atgriež true tikai kamēr revokācijas ieraksts nav expired. Šis dubultnieks nodrošina, ka redis loģika ir deterministiska un testējama lokāli, bez reāla redis servisa.

Kopējais ieguvums: Kombinējot EF Core InMemory (datubāzei) un FakeRedisService (redis loģikai), integrācijas testi kļūst neatkarīgi no ārējām sistēmām, tiek panākta datu izolācija un testu izpilde ir gan ātra, gan stabila. Tādēļ testu rezultāti atspoguļo tieši lietojumprogrammas loģikas pareizību, nevis infrastruktūras stāvokli.

Autentifikācijas simulācija: testa vidē ģenerēti JWT tokeni, izmantojot JwtService, lai simulētu autorizētus pieprasījumus ar dažādām lomām. Automatizācija un nepārtrauktā integrācija (CI/CD): GitHub Actions workflow automātiski palaiž visu testu kopu katrā Pull Request un push notikumā, bloķējot integrāciju kļūdaina koda gadījumā. Šī procesa loģiskā shēma ir attēlota 4.2. attēlā.

Kā redzams shēmā, pēc koda iesniegšanas (Push/PR) paralēli tiek palaisti trīs neatkarīgi CI procesi: backend koda validācija un testēšana, frontend statiskā analīze un testēšana, kā arī drošības skenēšana ar Trivy rīku. Tikai veiksmīgu testu gadījumā CD fāzē tiek veikta jaunu Docker attēlu būvēšana, veco tagu tīrīšana GitHub Container Registry (GHCR) un automātiska izvietošana (Deploy). Pēc izvietošanas sistēma veic veselības pārbaudi (Health?); ja tā neizdodas, tiek palaists automatizēts atkata skripts (Rollback.sh).

### 4.3. Vienību testu rezultāti

Tika izstrādāti un veiksmīgi nokārtoti testi kritiskajiem sistēmas komponentiem:

1. PasswordHashingService – paroles šifrēšanas un verifikācijas algoritma (BCrypt) korektuma pārbaude, tostarp nepareizas paroles noraidīšana.
2. JwtService – JWT tokenu ģenerācijas, derīguma termiņa, issuer/audience validācijas un claims struktūras pārbaude.
3. EncryptionService – sensitīvu datu (e-pasts, telefona numurs) šifrēšanas/dešifrēšanas cikliskā pārbaude un NULL/tukšu virkņu droša apstrāde.
4. RateLimitingService – pieprasījumu ierobežošanas logikas validācija (atļauja zem sliekšņa, liegums pārsniegšanas gadījumā, loga izbeigšanās un atļaujas atjaunošana).

#### **4.4. Integrācijas un API endpointu testi**

Izmantojot WebApplicationFactory, tika simulēta pilna servera vide bez atkarības no ārējās infrastruktūras. Pārbaudīti šādi funkcionālie scenāriji:

1. Autentifikācija: reģistrācija ar unikālu e-pastu, dublikātu noraidīšana (400 Bad Request), pieteikšanās ar nepareizu paroli (401 Unauthorized), JWT/Refresh Token rotācija un droša izrakstīšanās (logout ar cookie tīrīšanu).
2. Lietotāju profili: autorizēta profila lasīšana/rediģēšana, piekļuves liegums citu lietotāju datu rediģēšanai (403 Forbidden), paroles maiņas plūsma ar iepriekšējās paroles verifikāciju.
3. Dzīvnieku un adopciju pārvaldība: CRUD operācijas ar lomām (Shelter), adopcijas pieteikuma izveide, dubultā pieteikuma bloķēšana, statusa maiņa (apstiprināšana/noraidīšana) ar tiesību pārbaudi.
4. IoT ierīču reģistrācija: MAC adrešu formātu validācija, dublikātu konflikta (409 Conflict) apstrāde, piekļuves robežu un DeviceUser API atslēgu ģenerācijas loģika.

#### **4.5. Drošības un veiktspējas aspekti**

1. Drošības validācija: CSRF token ģenerācijas/pārbaudes plūsma, CORS politikas ievērošana, JWT claimu verifikācija, Refresh Token revokācija caur Redis kešatmiņu.
2. Veiktspējas novērtējums: InMemory DB integrācijas testi demonstrē vidējo atbildes laiku < 100ms vienkāršiem CRUD pieprasījumiem. SignalR/Redis backplane nodrošina reāllaika datu piegādi bez bloķējošām sinhronām operācijām.

#### 4.6. Testu rezultātu kopsavilkums

Tika izstrādāti testi galveniem sistēmas komponentiem:

| <b>Modulis / Klase</b> | <b>Pārklājuma<br/>procents (%)</b> | <b>Pārbaudītā funkcionalitāte un loģika</b>                                     |
|------------------------|------------------------------------|---------------------------------------------------------------------------------|
| JwtService Config      | 100.0%                             | Autorizācijas pilnvaru (JWT) ģenerēšana,<br>parakstīšana un claims validācija   |
| UserEmailService       | 88.8%                              | Paziņojumu sistēmas, reģistrācijas apstiprinājumu<br>un e-pasta izsūtnes loģika |
| ShelterService         | 82.5%                              | Patversmju kontu pārvaldība, datu integritātes<br>pārbaude un CRUD operācijas   |
| AuthController         | 80.8%                              | Lietotāju reģistrācija, autentifikācija, lomu kontrole<br>un drošības plūsmas   |
| EncryptionService      | 75.3%                              | Sensitīvu datu (e-pasts, telefons) šifrēšanas un<br>dešifrēšanas cikli          |

Visi 73 izstrādātie testi tika izpildīti sekmīgi bez kļūdām (Succeeded: 73, Failed: 0). Testu pārklājuma vizuālais pārskats ir redzams 4.1. attēlā.

## SECINĀJUMI

1. Kvalifikācijas darba ietvaros izstrādāta funkcionāli pilnvērtīga programmatūras sistēma “Smart Shelter IoT”, kas efektīvi automatizē dzīvnieku patversmes pārvaldības procesus. Sistēma apvieno tīmekļa lietotni, servera daļu un IoT aparatūru vienotā, integrētā platformā.
2. Izstrādātā datubāzes arhitektūra izmanto poliglodu pieeju: PostgreSQL strukturētajiem datiem, MongoDB attēliem un IoT telemetrijai, Redis kešatmiņai un sesijām. Šāda pieeja nodrošina optimālu veikspēju dažādiem datu tiem un piekļuves paradumiem.
3. Realizētā IoT apakšsistēma, izmantojot ESP32D mikrokontrollerus un Sensirion SCD41 sensorus, nodrošina nepārtrauktu apkārtējās vides monitoringu. Dati tiek pārsūtīti caur HTTPS protokolu un vizualizēti reāllaikā SignalR tehnoloģijas palīdzību, ļaujot personālam savlaicīgi reaģēt uz novirzēm.
4. Daudzslāņu drošības arhitektūra (Cloudflare WAF, Nginx, JWT, HTTPS, RBAC, Rate Limiting) nodrošina sistēmas aizsardzību pret biežākajiem tīmekļa draudiem. Izstrādātā “Pet Compatibility Scorer” algoritma iekļaušana demonstrē inovatīvu pieeju adoptācijas procesa optimizācijai.
5. Automatizētā testēšana (xUnit, WebApplicationFactory), integrēta CI/CD cauruļvadā caur GitHub Actions, apstiprināja sistēmas uzticamību: visi testu kopskaits noslēdzās veiksmīgi ar nulli kļūdām izpildes laikā. Aptveres procents kritiskajiem komponentiem sasniedza augstu līmeni. Veiktā testēšana apliecina, ka “Smart Shelter IoT” sistēma atbilst augstiem uzticamības, drošības un veikspējas standartiem. Visi kritiskie biznesa procesi, autentifikācijas mehānismi, datu validācijas un IoT datu apstrādes plūsmas ir pilnībā validēti automatizētos testos. Nulles kļūdu skaits izpildes laikā un CI/CD integrācija nodrošina stabilu produkta attīstību, vieglu apkopi un pilnīgu gatavību izvietojumam ražošanas vidē.

## IESPĒJAMIE UZLABOJUMI

1. Integrēt AI/ML moduli dzīvnieku uzvedības analīzei, izmantojot uzkrātos IoT datus. Mašīnmācīšanās algoritmi varētu prognozēt dzīvnieku veselības novirzes, identificējot anomālijas sensoru rādītājos pirms tās kļūst kritiskas.
2. Paplašināt IoT sensoru tīklu, pievienojot svara sensorus (barošanas uzsākšanas kontrolei), trākša līmeņa sensorus un video novērošanas kameras ar dzīvnieku skaita atpazīšanu, ļaujot izveidot pārskatīgāku vides monitoringa sistēmu.
3. Izstrādāt mobilā lietotni (React Native vai Flutter) adoptētājiem un patversmes darbiniekiem, lai nodrošinātu ātrāku piekļuvi sistēmai un push paziņojumus kritisku vides parametru pārsniegšanas gadījumos.
4. Izveidot ziedojumu un labdarības moduli, kas ļautu lietotājiem veikt vienreizējus vai regulārus ziedojumus patversmes uzturēšanai. Sistēma varētu ietvert drošu maksājumu vārteju integrāciju (piem., Stripe, Paysera), caurskatāmu līdzekļu izlietojuma atskaišu ģenerēšanu un iespēju konkrētiem dzīvniekiem vai infrastruktūras uzlabojumiem novirzīt mērķtiecīgus ziedojumus.

## IZMANTOTĀS LITERATŪRAS UN AVOTU SARAKSTS

1. Microsoft. ASP.NET Core documentation. Microsoft Learn, 2024. Pieejams: <https://learn.microsoft.com/en-us/aspnet/core> [skatīts 2025. gada 10. aprīlī].
2. Meta Platforms. React – The library for web and native user interfaces. React documentation, 2024. Pieejams: <https://react.dev> [skatīts 2025. gada 12. aprīlī].
3. The PostgreSQL Global Development Group. PostgreSQL 16 Documentation. 2024. Pieejams: <https://www.postgresql.org/docs/16/> [skatīts 2025. gada 15. aprīlī].
4. Espressif Systems. ESP32 Technical Reference Manual. Version 5.1. 2024. Pieejams: [https://www.espressif.com/sites/default/files/documentation/esp32\\_technical\\_reference\\_manual\\_en.pdf](https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf) [skatīts 2025. gada 18. aprīlī].
5. Docker Inc. Docker Documentation. 2024. Pieejams: <https://docs.docker.com> [skatīts 2025. gada 22. aprīlī].
6. Sensirion AG. SCD4x Datasheet – CO2 Sensor with Temperature and Humidity. 2023. Pieejams: [https://sensirion.com/media/documents/48C4B7FB/64C134E7/Sensirion\\_SCD4x\\_Datasheet.pdf](https://sensirion.com/media/documents/48C4B7FB/64C134E7/Sensirion_SCD4x_Datasheet.pdf) [skatīts 2025. gada 5. maijā].
7. Microsoft. SignalR for ASP.NET Core. Microsoft Learn, 2024. Pieejams: <https://learn.microsoft.com/en-us/aspnet/core/signalr/introduction> [skatīts 2025. gada 8. maijā].
8. MongoDB Inc. MongoDB Documentation. 2024. Pieejams: <https://www.mongodb.com/docs/> [skatīts 2025. gada 20. aprīlī].
9. Redis Ltd. Redis Documentation. 2024. Pieejams: <https://redis.io/docs/> [skatīts 2025. gada 22. aprīlī].
10. Internet Engineering Task Force. RFC 7519 – JSON Web Token (JWT). IETF, 2015. Pieejams: <https://www.rfc-editor.org/rfc/rfc7519> [skatīts 2025. gada 25. aprīlī].
11. Provos, N.; Mazières, D. A Future-Adaptable Password Scheme. USENIX Annual Technical Conference, 1999. Pieejams: <https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf> [skatīts 2025. gada 28. aprīlī].
12. Internet Engineering Task Force. RFC 9110 – HTTP Semantics (HTTPS/TLS). IETF, 2022. Pieejams: <https://www.rfc-editor.org/rfc/rfc9110> [skatīts 2025. gada 1. maijā].
13. GitHub. GitHub Actions Documentation. 2024. Pieejams: <https://docs.github.com/en/actions> [skatīts 2025. gada 3. maijā].

14. Nginx Inc. NGINX Documentation. 2024. Pieejams: <https://nginx.org/en/docs/> [skatīts 2025. gada 6. maijā].

## **GALVOJUMS**

Ar šo, es, \_\_\_\_\_, galvoju, ka kvalifikācijas darbs ir izpildīts patstāvīgi, bez citu palīdzības, no svešiem pirmavotiem ņemtie dati un definējumi ir uzrādīti darbā. Šis darbs nekādā veidā nav iesniegts nevienai citai pārbaudījuma komisijai.

202\_.gada \_\_\_\_.

---

(paraksts)

## NOVĒRTĒJUMS

Kvalifikācijas darbs aizstāvēts **Profesionālās vidusskolas „Victoria” pārbaudījuma komisijas** sēdē 202\_.gada\_.\_\_\_\_\_ un novērtēts ar atzīmi \_\_\_\_\_.

**Komisijas priekšsēdētājs** \_\_\_\_\_  
vārds, uzvārds paraksts